

**LAPORAN PENGUJIAN SIMULASI CACHING PADA
SCALABLE SYSTEM MENGGUNAKAN PYTHON DAN
LOCALHOST**

Dosen Pengampuh Mata Kuliah : RUNAL REZKIAWAN S.Kom.,M.T



DISUSUN OLEH :

NAMA : NURDIAN

NIM : 105841118923

KELAS : 6A

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR**

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari pembelajaran mengenai Scalable System Design, khususnya pada materi Strategi Caching, Memcached/Redis Concept, CDN, serta Cache Invalidation/Eviction.

Tujuan penyusunan laporan ini adalah untuk memahami konsep caching, cara kerja cache hit dan cache miss, serta implementasi sederhana strategi Cache Aside Pattern menggunakan bahasa pemrograman Python dan localhost. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang.

Makassar, Juni 2026

Nurdian

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	1
1.3 Tujuan	1
1.4 Manfaat.....	2
1.5 Batasan Masalah	2
BAB II	3
LANDASAN TEORI	3
2.1 Pengerian Caching.....	3
2.2 Konsep Dasar Cache Hit dan Cache Miss	3
2.3 Strategi Cache Aside Pattern	4
2.4 Peran Caching Dalam Scalable System.....	4
BAB III	5
METODE DAN IMPLEMENTASI	5
3.1 Metode Pengujian.....	5
3.2 Implementasi Sistem.....	5
3.3 Lingkungan Pengujian.....	5
BAB IV	7
HASIL DAN PEMBAHASAN	7
4.1 Hasil Pengujian Simulasi.....	7
4.2 Pembahasan Efesiensi Performa.....	8
BAB V	9
PENUTUP	9
5.1 Kesimpulan.....	9

5.2 Saran	9
DAFTAR PUSTAKA	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menyebabkan peningkatan jumlah pengguna dan permintaan terhadap aplikasi berbasis web maupun mobile. Semakin banyak pengguna yang mengakses suatu sistem, semakin besar pula beban yang harus ditangani oleh server dan database. Jika setiap permintaan pengguna selalu diproses langsung ke database, maka performa sistem dapat menurun dan waktu respons menjadi lebih lambat.

Salah satu solusi yang digunakan dalam scalable system adalah caching. Caching merupakan teknik penyimpanan sementara data yang sering diakses sehingga data tersebut dapat digunakan kembali tanpa harus mengambilnya dari database. Dengan adanya caching, waktu respons menjadi lebih cepat, beban server berkurang, dan pengalaman pengguna menjadi lebih baik.

Pada laporan ini dilakukan simulasi sederhana menggunakan Python dan localhost untuk memahami bagaimana caching bekerja melalui konsep cache hit dan cache miss serta implementasi strategi Cache Aside Pattern.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan caching pada scalable system?
2. Bagaimana cara kerja cache hit dan cache miss?
3. Bagaimana implementasi strategi Cache Aside Pattern?
4. Bagaimana pengaruh caching terhadap performa sistem?

1.3 Tujuan

1. Memahami konsep dasar caching.
2. Mengetahui mekanisme cache hit dan cache miss.
3. Mengimplementasikan strategi Cache Aside Pattern menggunakan Python.
4. Menganalisis pengaruh caching terhadap performa sistem.

1.4 Manfaat

1. Menambah pemahaman mengenai scalable system.
2. Memahami peran caching dalam meningkatkan performa aplikasi.
3. Menjadi dasar dalam implementasi Redis atau Memcached pada sistem nyata.
4. Mengetahui cara mengurangi beban database melalui caching.

1.5 Batasan Masalah

1. Pengujian dilakukan menggunakan Python dan Flask.
2. Simulasi dijalankan pada localhost.
3. Cache menggunakan penyimpanan sederhana berupa dictionary Python.
4. Pengujian hanya membahas Cache Aside Pattern.
5. Tidak menggunakan Redis atau Memcached secara langsung.

BAB II

LANDASAN TEORI

2.1 Pengerian Caching

Caching adalah teknik penyimpanan sementara data yang sering digunakan agar dapat diakses kembali dengan lebih cepat tanpa harus mengambil data dari sumber utama seperti database atau server. Data yang disimpan pada cache biasanya berada di memori sehingga proses akses berlangsung lebih cepat dibandingkan akses langsung ke database.

2.2 Konsep Dasar Cache Hit dan Cache Miss

Cache Hit

Cache hit terjadi ketika data yang diminta pengguna sudah tersedia di dalam cache. Sistem dapat langsung mengirimkan data tersebut tanpa perlu mengakses database.

Keuntungan cache hit:

- Waktu respons lebih cepat.
- Mengurangi beban database.
- Meningkatkan efisiensi sistem.

Cache Miss

Cache miss terjadi ketika data yang diminta belum tersedia di cache. Sistem harus mengambil data dari database terlebih dahulu, kemudian menyimpannya ke cache untuk digunakan pada permintaan berikutnya.

Proses cache miss:

1. User mengirim request.
2. Sistem memeriksa cache.
3. Data tidak ditemukan.
4. Sistem mengambil data dari database.
5. Data disimpan ke cache.
6. Data dikirim ke pengguna.

2.3 Strategi Cache Aside Pattern

Cache Aside Pattern merupakan strategi caching yang paling umum digunakan. Pada strategi ini aplikasi akan memeriksa cache terlebih dahulu sebelum mengakses database.

Langkah kerja:

1. Aplikasi mengecek cache.
2. Jika data ditemukan, data langsung dikembalikan (cache hit).
3. Jika data tidak ditemukan, aplikasi mengambil data dari database.
4. Data hasil query disimpan ke cache.
5. Data dikirim ke pengguna.

Kelebihan:

- Mudah diterapkan.
- Mengurangi beban database.
- Cocok untuk data yang sering dibaca.

Kekurangan:

- Risiko data cache tidak sinkron dengan database.

2.4 Peran Caching Dalam Scalable System

Caching memiliki peran penting dalam scalable system karena mampu:

- Mengurangi jumlah query ke database.
- Mempercepat waktu respons aplikasi.
- Mengurangi penggunaan sumber daya server.
- Meningkatkan kapasitas sistem dalam menangani banyak pengguna.
- Meningkatkan pengalaman pengguna.

BAB III

METODE DAN IMPLEMENTASI

3.1 Metode Pengujian

Metode yang digunakan adalah simulasi caching menggunakan Python dan Flask pada localhost.

Sistem terdiri dari:

1. Browser sebagai client.
2. Flask sebagai server aplikasi.
3. Database simulasi berupa dictionary Python.
4. Cache simulasi berupa dictionary Python.

3.2 Implementasi Sistem

Alur kerja sistem:

1. User mengakses endpoint localhost.
2. Sistem memeriksa cache.
3. Jika data tersedia maka terjadi cache hit.
4. Jika data tidak tersedia maka terjadi cache miss.
5. Sistem mengambil data dari database.
6. Data disimpan ke cache.
7. Data dikirim kembali ke pengguna.

Arsitektur sederhana:

User → Flask Server → Cache → Database

3.3 Lingkungan Pengujian

Perangkat Lunak:

- Windows 10/11
- Python 3.x
- Flask Framework
- Browser Google Chrome

Perangkat Keras:

- Laptop/PC
- RAM minimal 4 GB

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Pengujian Simulasi

Pengujian Pertama

`http://127.0.0.1:5000/produk`

Hasil

 **CACHE MISS**

Laptop Asus AsusVivoBook

Data diambil dari database

Data kemudian disimpan ke cache

Karena data belum tersedia pada cache sehingga sistem mengambil data dari database terlebih dahulu.

Waktu akses:

± 3 detik

Pengujian Kedua

User melakukan refresh halaman.

Hasil

 **CACHE HIT**

Laptop Asus AsusVivoBook

Data diambil dari cache

Waktu akses: Sangat Cepat

Data telah tersedia pada cache sehingga sistem langsung mengirimkan data tanpa mengakses database.

Waktu akses:

± 0 detik

Pengujian Ketiga

`http://localhost:5000/clear-cache`

Cache dihapus.

Kemudian mengakses kembali:

`http://127.0.0.1:5000/produk`

4.2 Pembahasan Efisiensi Performa

Berdasarkan hasil pengujian, dapat dilihat bahwa penggunaan cache memberikan peningkatan performa yang signifikan.

Perbandingan hasil:

Kondisi	Waktu Akses
Cache Miss	± 3 detik
Cache	± 0 detik

Pada kondisi cache miss, sistem membutuhkan waktu lebih lama karena harus mengambil data dari database. Sedangkan pada cache hit, data langsung diambil dari cache yang berada di memori sehingga proses menjadi jauh lebih cepat.

Hal ini membuktikan bahwa caching mampu:

- Mengurangi waktu respons.
- Mengurangi beban database.
- Meningkatkan efisiensi sistem.
- Mendukung skalabilitas aplikasi.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian, dapat disimpulkan bahwa caching merupakan teknik yang efektif untuk meningkatkan performa sistem. Implementasi Cache Aside Pattern berhasil menunjukkan mekanisme cache hit dan cache miss pada aplikasi berbasis Python dan Flask.

Hasil pengujian menunjukkan bahwa cache hit memberikan waktu respons yang jauh lebih cepat dibandingkan cache miss karena data tidak perlu diambil kembali dari database. Dengan demikian, caching dapat mengurangi beban server, meningkatkan efisiensi penggunaan sumber daya, serta mendukung pengembangan scalable system.

5.2 Saran

1. Pengembangan selanjutnya dapat menggunakan Redis atau Memcached sebagai cache yang sesungguhnya.
2. Pengujian dapat dilakukan dengan jumlah request yang lebih besar untuk melihat peningkatan performa secara lebih detail.
3. Implementasi dapat dikombinasikan dengan CDN untuk meningkatkan distribusi konten.
4. Dapat dilakukan pengujian strategi caching lain seperti Write Through, Write Back, dan Read Through untuk membandingkan performanya.

DAFTAR PUSTAKA

- Cloudflare. (2025). *What is caching?* Cloudflare Learning Center.
<https://www.cloudflare.com/learning/cdn/what-is-caching/>
- Google Cloud. (2024). *Caching overview.* Google Cloud.
<https://cloud.google.com/architecture/caching-overview>
- Memcached Project. (2025). *Memcached documentation.* <https://memcached.org/>
- Microsoft. (2024). *Cache-aside pattern.* Microsoft Learn.
<https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside>
- Redis Ltd. (2025). *Redis documentation.* <https://redis.io/docs/>